

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-23T08:27:55.5502558Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260923-082755551.prof
Profil mémoire	memory-20260923-082755551.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	554.57 ms	554570487	3870797	24

Détail hyperfine

Métrique	Valeur
Moyenne	63.5702 s
Écart-type	n/a
Médiane	63.5702 s
Temps utilisateur	61.6034 s
Temps système	1.9453 s
Min	63.5702 s
Max	63.5702 s

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **59.56s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).shouldInfect	38s	63.80%	58.76s	98.66%
jeu-de-la-vie/game.(*Board).coordinates (inlin	11.36s	19.07%	11.36s	19.07%
jeu-de-la-vie/game.squaredDistance (inline)	9.37s	15.73%	9.37s	15.73%
jeu-de-la-vie/game.(*Board).Step	0.19s	0.32%	59.16s	99.33%
jeu-de-la-vie/game.BenchmarkTick	0	0.00%	59.16s	99.33%
testing.(*B).launch	0	0.00%	59.16s	99.33%

testing.(*B).runN	0	0.00%	59.16s	99.33%
-------------------	---	-------	--------	--------

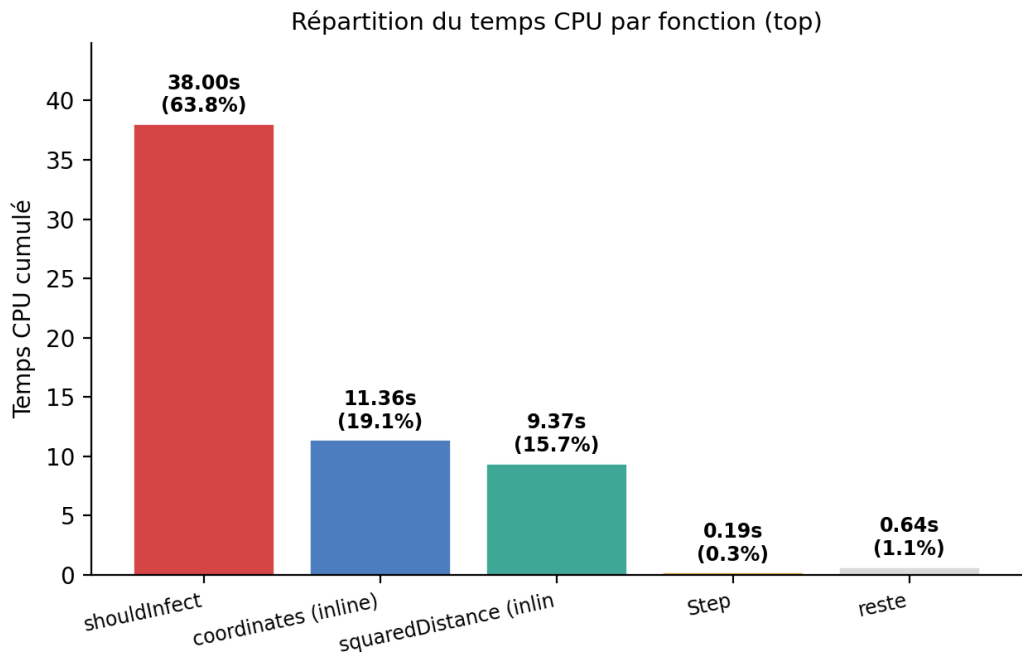


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

(Graphe d'appel visuel non généré — Graphviz/'dot' introuvable dans le PATH. Installez Graphviz pour l'obtenir automatiquement au prochain run.)

2.1 Diagnostic automatique

La fonction dominante de ce run est `jeu-de-la-vie/game.(*Board).shouldInfect`, responsable de **63.8%** du temps CPU total (flat), pour **98.7%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

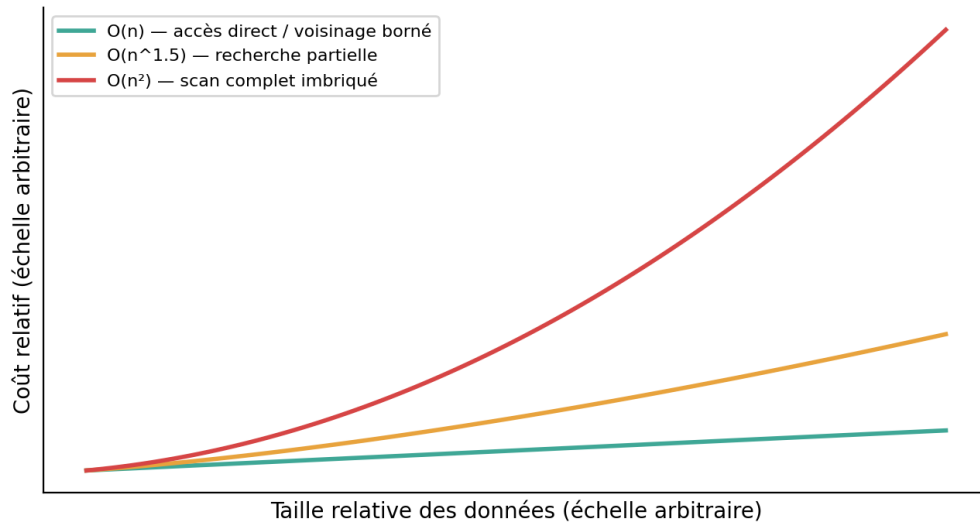


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : **alloc_space**. Total : **383.46MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).Step	371.41MB	96.86%	371.41MB	96.86%
jeu-de-la-vie/game.RandomBoard	5.52MB	1.44%	6.91MB	1.80%
runtime.mallocgc	2MB	0.52%	2MB	0.52%
jeu-de-la-vie/game.BenchmarkTick	0	0.00%	378.82MB	98.79%
runtime.newobject	0	0.00%	2MB	0.52%
testing.(*B).launch	0	0.00%	374.17MB	97.58%
testing.(*B).run1.func1	0	0.00%	5.15MB	1.34%
testing.(*B).runN	0	0.00%	379.32MB	98.92%

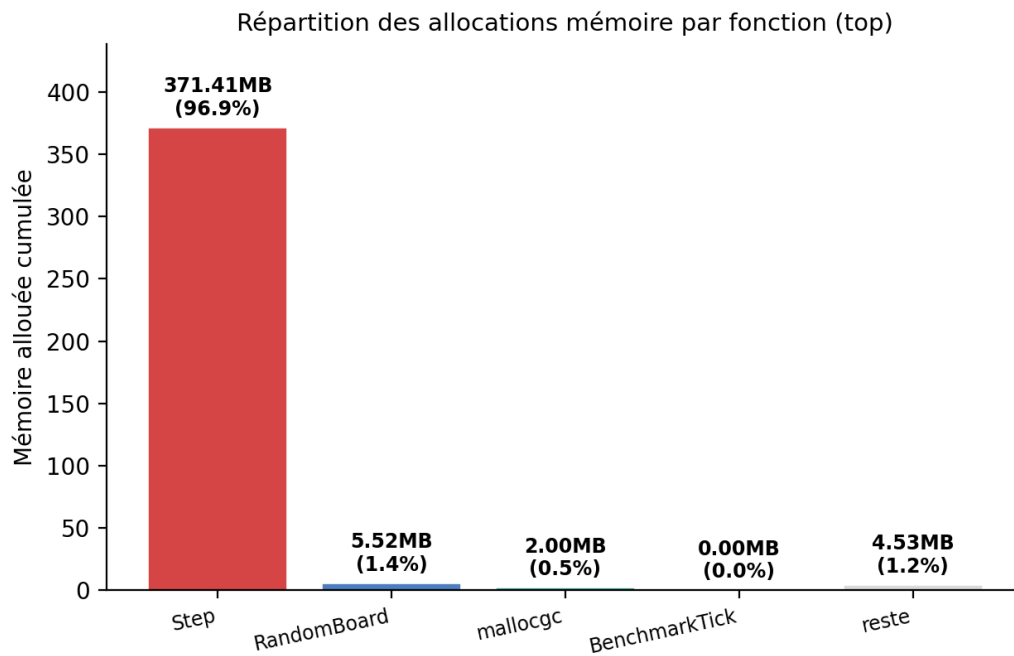


Figure — Répartition des allocations mémoire par fonction.

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.
- **Pas de trace GC détaillée** (GODEBUG=gctrace=1) — utile si les allocations mémoire (section 3) sont significatives.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600 Å— 600.
- La carte et le gÃ©nÃ©rateur alÃ©atoire utilisent toujours la seed 42.
- La latence API est mesurÃ©e Å part dans le frontend, sans rendu canvas.
- Les mÃ©triques CPU bas niveau, GC dÃ©taillÃ©, I/O, rÃ©seau et base de donnÃ©es ne sont pas instrumentÃ©es dans ce benchmark.

Rapport gÃ©nÃ©rÃ© automatiquement le 2026-09-23 10:30:03 par generate_benchmark_report.py, Å partir des fichiers les plus rÃ©cents du dossier de benchmarks. Analyse rÃ©alisÃ©e avec go tool pprof.